spatempBFA Toy Example Illustration

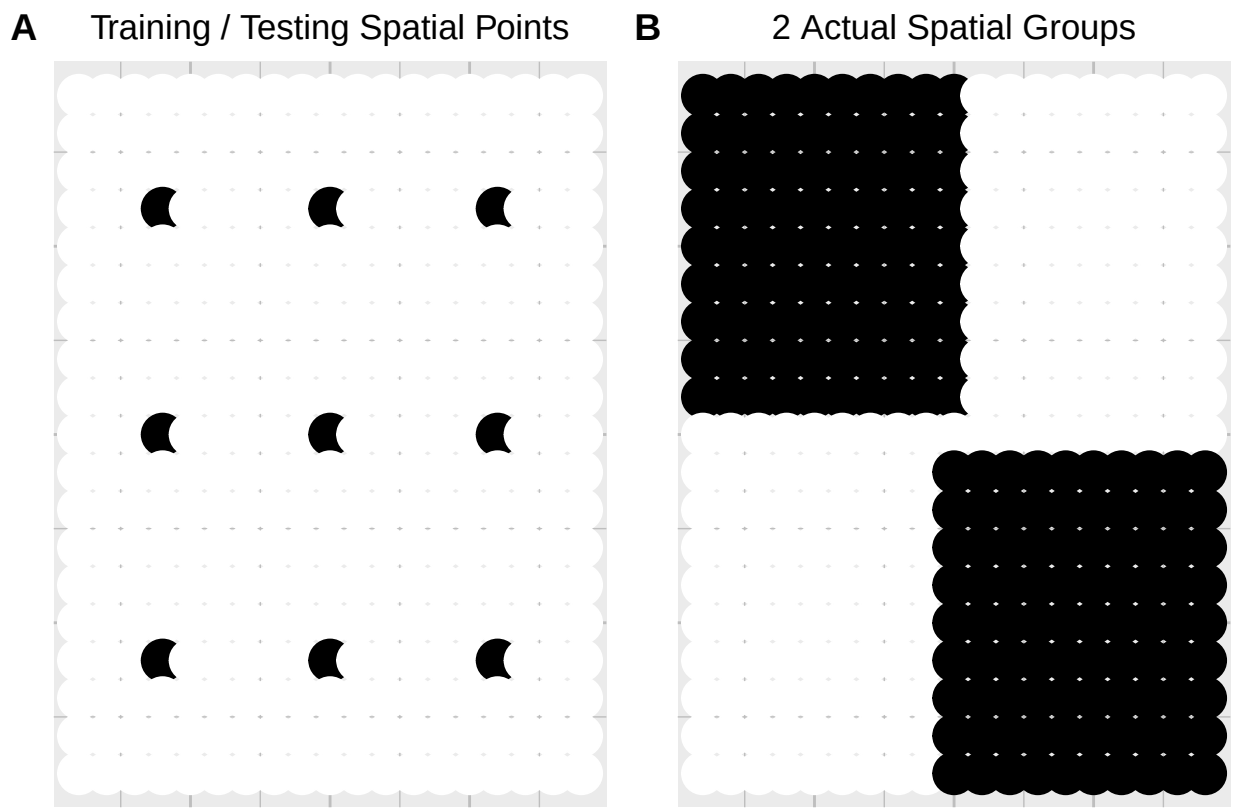
Yifan CHENG y.cheng@u.nus.edu

2024-07-20

Data Generation and Model Fitting

```
rm(list=ls())
library(mvtnorm)
library(fields)
library(tidyverse)
library(ggpubr)
numSpatOverallGroups <- 2
M <- 361
m = 352 # 9 testing spatial points
K <- 2
O <- 1
L <- min(10, M)
LjVec <- rep(min(10, M), K)
sqrootM <- 19
Nu <- 60
trainingT <- 50 # training set T = 50
testingT <- 10
Time <- 1:trainingT
TimeDist <- as.matrix(dist(1:60))
APsi = 0.1; BPsi = 4.5
set.seed(29)
calcGroup <- function(row, col, sqrootM){
  return((row - 1)*sqrootM + col)
}
sigma2 <- 0.01 # actual sigma^2(i,o) (for i=1,2,...,M and o=1) values
psi <- 2.3
kappa <- 0.7
tempMat <- matrix(runif(K*K,0,1), K, K)
Upsilon <- t(tempMat)%*%tempMat
rho <- 0.8
D <- rdist(expand.grid(1:sqrootM, 1:sqrootM))
Frho <- exp(-rho*D)
Hpsi <- exp(-psi*TimeDist)
Eta <- rmvnorm(1, mean=rep(0, Nu*K), sigma=kronecker(Hpsi, Upsilon)) # actual Eta (c(Eta_1,...,Eta_T))
Lambda <- matrix(5, M * O, K)
theta2j <- c(10, -10)
rowColInd <- rbind(expand.grid(1:10, 1:9), expand.grid(10:19, 11:19))
whichGroup1 <- mapply(calcGroup, rowColInd[,2], rowColInd[,1], sqrootM = sqrootM)
spatGroupOverall <- rep(2,M); spatGroupOverall[whichGroup1] <- 1
# matrix(spatGroupOverall, 19, 19)
```

```r
Lambda[,2] = theta2j[spatGroupOverall]
Sigma.NuMO <- matrix(rnorm(Nu * M * O, sd = sqrt(sigma2)), M*O, Nu)
EtaMat <- matrix(Eta, K, Nu)
meanMat <- Lambda%*%EtaMat #M*O\times Nu
YtrainingTemp <- as.vector(meanMat[,1:trainingT] + Sigma.NuMO[,1:trainingT])
YtestingTemp <- as.vector(meanMat[,(trainingT+1):Nu] + Sigma.NuMO[,(trainingT+1):Nu])
testingRowCol <- expand.grid(c(4, 10, 16), c(4, 10, 16))
whichTesting <- mapply(calcGroup, testingRowCol[,2], testingRowCol[,1], sqrootM = sqrootM)
Dtraining <- as.matrix(D[-whichTesting, -whichTesting])
discardInd <- vector()
for(whichTestingLoc in whichTesting){
  discardInd <- c(discardInd, seq(from = whichTestingLoc, by = M, length.out = trainingT))
}
Ytraining <- YtrainingTemp[-discardInd]
YtestingSpat <- YtrainingTemp[discardInd]
rm(YtrainingTemp)
spatGroupOverallTraining <- spatGroupOverall[-whichTesting]
distOrigNew = as.matrix(D[-whichTesting, whichTesting])
distNewNew = as.matrix(D[whichTesting, whichTesting])
dat = data.frame(Y = Ytraining); dist = Dtraining
tempTestingDiscardInd <- vector()
for(whichTestingLoc in whichTesting){
  tempTestingDiscardInd <- c(tempTestingDiscardInd,
                             seq(from = whichTestingLoc, by = M, length.out = testingT))
}
YtestingTemp <- YtestingTemp[-tempTestingDiscardInd]
xcoord <- rep(1:sqrootM, sqrootM)
ycoord <- rep(seq(sqrootM, 1, by = -1), each = sqrootM)
trainingTestingSpatGp <- rep(1, M); trainingTestingSpatGp[whichTesting] = 2
spatGpDF <- data.frame(x = xcoord, y = ycoord, actualGp <- as.factor(spatGroupOverall),
                       trainingTestingSpatGp <- as.factor(trainingTestingSpatGp))
trainingTestingPlotBW <- ggplot(spatGpDF) + ggtitle("Training / Testing Spatial Points") +
  geom_point(aes(x = x, y = y, col = trainingTestingSpatGp), size = 7, show.legend = FALSE) +
  scale_color_manual(values = c("white", "black")) + labs(x = "", y = "") +
  theme(plot.title = element_text(hjust = 0.5),
        axis.text.x = element_blank(), axis.text.y = element_blank(),
        axis.ticks.x = element_blank(), axis.ticks.y = element_blank(),
        panel.grid.major = element_line(color="grey"),
        panel.grid.minor = element_line(color="grey"))
actualSpatPlotBW <- ggplot(spatGpDF) + ggtitle("2 Actual Spatial Groups") +
  geom_point(aes(x = x, y = y, col = actualGp), size = 7, show.legend = FALSE) +
  scale_color_manual(values = c("black", "white")) + labs(x = "", y = "") +
  theme(plot.title = element_text(hjust = 0.5),
        axis.text.x = element_blank(), axis.text.y = element_blank(),
        axis.ticks.x = element_blank(), axis.ticks.y = element_blank(),
        panel.grid.major = element_line(color="grey"),
        panel.grid.minor = element_line(color="grey"))
ggarrange(trainingTestingPlotBW, actualSpatPlotBW, align = "h", ncol = 2, nrow = 1,
          labels = c("A", "B"))
```



The entire simulated data set consists of $y_t(\boldsymbol{s}_i)$'s from 60 time points and 361 spatial locations, which belong to 2 actual spatial groups (**B**). 352 out of the 361 locations (**A**) are training ones, and the 9 remaining black locations (**A**) are testing ones. The training data set used to fit all 6 models (our 4 models `fullGPfixedL`, `NNGPblockFixedL`, `NNGPsequenFixedL`, and `NNGPsequenVaryLj` & `spBFAL10`, `spBFALInf` from the R package `spBFA`) are data corresponding to the 352 white locations (**A**) and the first 50 time points. $y_t(\boldsymbol{s}_i)$'s corresponding to the 9 testing locations for the first 50 time points are used to assess our 4 models' spatial prediction performances. Similarly, $y_t(\boldsymbol{s}_i)$'s corresponding to the 352 training locations for the last 10 time points are used to assess all 6 models' temporal prediction performances.

```
library(spatempBFA)
library(coda)
MCMC <- list(NBurn = 40000, NSims = 10000, NThin = 2, NPilot = 5)
regFixedL.simu <- bfaFixedL(Y ~ 0, data = dat, dist = Dtraining, time = Time,  K = K,
                            starting = NULL, hypers = NULL, tuning = NULL,
                            mcmc = MCMC,
                            L = L,
                            family = "normal",
                            temporal.structure = "exponential",
                            spatial.structure = "continuous",
                            seed = 29,
                            gamma.shrinkage = TRUE,
                            include.time = TRUE,
                            include.space = TRUE,
                            clustering = TRUE,
                            seasonPeriod = 1,
                            equalTimeDist = TRUE,
                            spatApprox = FALSE,
```

```r
                            alphaMethod = "block",
                            h = 15,
                            storeSpatPredPara = TRUE,
                            storeWeights = TRUE,
                            alphasWeightsToFiles = FALSE)
save(regFixedL.simu, file="toyExfullGPfixedL.RData")
GibbsStepTimeFixedLfullGP <- regFixedL.simu$GibbsStepTime
save(GibbsStepTimeFixedLfullGP, file = "GibbsStepTimeFixedLfullGP.RData")
Diags <- diagnostics(regFixedL.simu, diags = c("dic", "dinf", "meanIC", "waic"),
                     keepDeviance = TRUE)
save(Diags, file = "toyExfullGPfixedLDiags.RData")
Deviance <- as.mcmc(Diags$deviance)
save(Deviance, file = "toyExfullGPfixedLDeviance.RData")
spatpredFixedL <- predictNewLocFixedL(regFixedL.simu, 9, distOrigNew, distNewNew,
                                      NewX = NULL, NewTrials = NULL, Verbose = TRUE,
                                      seed = 29)
save(spatpredFixedL, file = "toyExNNGPspatpredFixedL.RData")
tempppredFixedL <- predictNewTime(regFixedL.simu, (trainingT+1):Nu, seed = 29)
save(tempppredFixedL, file = "toyExNNGPtempppredFixedL.RData")
fittedClusGpMat <- matrix(0, m, 3)
clusFixedL10 <- clusteringFixedL(regFixedL.simu, o = 1, nkeep = 10,
                                 nCent = numSpatOverallGroups)
fittedClusGpMat[,1] <- clusFixedL10$cluster
clusFixedL100 <- clusteringFixedL(regFixedL.simu, o = 1, nkeep = 100,
                                  nCent = numSpatOverallGroups)
fittedClusGpMat[,2] <- clusFixedL100$cluster
clusFixedL1000 <- clusteringFixedL(regFixedL.simu, o = 1, nkeep = 1000,
                                   nCent = numSpatOverallGroups)
fittedClusGpMat[,3] <- clusFixedL1000$cluster
weightsNumIter <- c(10, 100, 1000)
colnames(fittedClusGpMat) <- paste(weightsNumIter, "iterWeights")
save(fittedClusGpMat, file = "fullGPfixedLtoyExfittedClusGpMat.RData")
regFixedL.simu.block <- bfaFixedL(Y ~ 0, data = dat, dist = Dtraining, time = Time,
                                  K = K,
                                  starting = NULL, hypers = NULL, tuning = NULL,
                                  mcmc = MCMC,
                                  L = L,
                                  family = "normal",
                                  temporal.structure = "exponential",
                                  spatial.structure = "continuous",
                                  seed = 29,
                                  gamma.shrinkage = TRUE,
                                  include.time = TRUE,
                                  include.space = TRUE,
                                  clustering = TRUE,
                                  seasonPeriod = 1,
                                  equalTimeDist = TRUE,
                                  spatApprox = TRUE,
                                  alphaMethod = "block",
                                  h = 15,
                                  storeSpatPredPara = TRUE,
                                  storeWeights = TRUE,
                                  alphasWeightsToFiles = FALSE)
```

```r
save(regFixedL.simu.block, file="toyExNNGPblockFixedL.RData")
GibbsStepTimeFixedLblock <- regFixedL.simu.block$GibbsStepTime
save(GibbsStepTimeFixedLblock, file = "GibbsStepTimeFixedLblock.RData")
Diags.block <- diagnostics(regFixedL.simu.block,
                           diags = c("dic", "dinf", "meanIC", "waic"),
                           keepDeviance = TRUE)
save(Diags.block, file = "toyExNNGPblockFixedLDiags.RData")
Deviance.block <- as.mcmc(Diags.block$deviance)
save(Deviance.block, file = "toyExNNGPblockFixedLDeviance.RData")
spatpredFixedLblock <- predictNewLocFixedL(regFixedL.simu.block, 9,
                                           distOrigNew, distNewNew,
                                           NewX = NULL, NewTrials = NULL, Verbose = TRUE,
                                           seed = 29)
save(spatpredFixedLblock, file = "toyExNNGPspatpredFixedLblock.RData")
tempppredFixedLblock <- predictNewTime(regFixedL.simu.block, (trainingT+1):Nu, seed = 29)
save(tempppredFixedLblock, file = "toyExNNGPtempppredFixedLblock.RData")
fittedClusGpMat <- matrix(0, m, 3)
clusFixedL10 <- clusteringFixedL(regFixedL.simu.block, o = 1, nkeep = 10,
                                 nCent = numSpatOverallGroups)
fittedClusGpMat[,1] <- clusFixedL10$cluster
clusFixedL100 <- clusteringFixedL(regFixedL.simu.block, o = 1, nkeep = 100,
                                  nCent = numSpatOverallGroups)
fittedClusGpMat[,2] <- clusFixedL100$cluster
clusFixedL1000 <- clusteringFixedL(regFixedL.simu.block, o = 1, nkeep = 1000,
                                   nCent = numSpatOverallGroups)
fittedClusGpMat[,3] <- clusFixedL1000$cluster
weightsNumIter <- c(10, 100, 1000)
colnames(fittedClusGpMat) <- paste(weightsNumIter, "iterWeights")
save(fittedClusGpMat, file = "NNGPblockFixedLtoyExfittedClusGpMat.RData")
regFixedL.simu.sequen <- bfaFixedL(Y ~ 0, data = dat, dist = Dtraining, time = Time,
                                   K = K,
                                   starting = NULL, hypers = NULL, tuning = NULL,
                                   mcmc = MCMC,
                                   L = L,
                                   family = "normal",
                                   temporal.structure = "exponential",
                                   spatial.structure = "continuous",
                                   seed = 29,
                                   gamma.shrinkage = TRUE,
                                   include.time = TRUE,
                                   include.space = TRUE,
                                   clustering = TRUE,
                                   seasonPeriod = 1,
                                   equalTimeDist = TRUE,
                                   spatApprox = TRUE,
                                   alphaMethod = "sequential",
                                   h = 15,
                                   storeSpatPredPara = TRUE,
                                   storeWeights = TRUE,
                                   alphasWeightsToFiles = FALSE)
save(regFixedL.simu.sequen, file="toyExNNGPsequenFixedL.RData")
GibbsStepTimeFixedLsequen <- regFixedL.simu.sequen$GibbsStepTime
save(GibbsStepTimeFixedLsequen, file = "GibbsStepTimeFixedLsequen.RData")
```

```r
Diags.sequen <- diagnostics(regFixedL.simu.sequen,
                            diags = c("dic", "dinf", "meanIC", "waic"),
                            keepDeviance = TRUE)
save(Diags.sequen, file = "toyExNNGPsequenFixedLDiags.RData")
Deviance.sequen <- as.mcmc(Diags.sequen$deviance)
save(Deviance.sequen, file = "toyExNNGPsequenFixedLDeviance.RData")
spatpredFixedLsequen <- predictNewLocFixedL(regFixedL.simu.sequen, 9,
                                            distOrigNew, distNewNew,
                                            NewX = NULL, NewTrials = NULL, Verbose = TRUE,
                                            seed = 29)
save(spatpredFixedLsequen, file = "toyExNNGPspatpredFixedLsequen.RData")
tempppredFixedLsequen <- predictNewTime(regFixedL.simu.sequen, (trainingT+1):Nu, seed = 29)
save(tempppredFixedLsequen, file = "toyExNNGPtempppredFixedLsequen.RData")
fittedClusGpMat <- matrix(0, m, 3)
clusFixedL10 <- clusteringFixedL(regFixedL.simu.sequen, o = 1, nkeep = 10,
                                 nCent = numSpatOverallGroups)
fittedClusGpMat[,1] <- clusFixedL10$cluster
clusFixedL100 <- clusteringFixedL(regFixedL.simu.sequen, o = 1, nkeep = 100,
                                  nCent = numSpatOverallGroups)
fittedClusGpMat[,2] <- clusFixedL100$cluster
clusFixedL1000 <- clusteringFixedL(regFixedL.simu.sequen, o = 1, nkeep = 1000,
                                   nCent = numSpatOverallGroups)
fittedClusGpMat[,3] <- clusFixedL1000$cluster
weightsNumIter <- c(10, 100, 1000)
colnames(fittedClusGpMat) <- paste(weightsNumIter, "iterWeights")
save(fittedClusGpMat, file = "NNGPsequenFixedLtoyExfittedClusGpMat.RData")
regVaryLj.simu.sequen <- bfaVaryingLjs(Y ~ 0, data = dat, dist = Dtraining, time = Time,
                                       K = K,
                                       starting = NULL, hypers = NULL, tuning = NULL,
                                       mcmc = MCMC,
                                       LjVec = LjVec,
                                       family = "normal",
                                       temporal.structure = "exponential",
                                       spatial.structure = "continuous",
                                       seed = 29,
                                       gamma.shrinkage = TRUE,
                                       include.time = TRUE,
                                       include.space = TRUE,
                                       seasonPeriod = 1,
                                       equalTimeDist = TRUE,
                                       spatApprox = TRUE,
                                       alphaSequen = TRUE,
                                       h = 15,
                                       storeSpatPredPara = TRUE,
                                       storeWeights = TRUE)
save(regVaryLj.simu.sequen, file="toyExNNGPsequenVaryLj.RData")
GibbsStepTimeVaryLjSequen <- regVaryLj.simu.sequen$GibbsStepTime
save(GibbsStepTimeVaryLjSequen, file = "GibbsStepTimeVaryLjSequen.RData")
Diags.sequenVaryLj <- diagnostics(regVaryLj.simu.sequen,
                                  diags = c("dic", "dinf", "meanIC", "waic"),
                                  keepDeviance = TRUE)
save(Diags.sequenVaryLj, file = "toyExNNGPsequenVaryLjDiags.RData")
Deviance.sequenVaryLj <- as.mcmc(Diags.sequenVaryLj$deviance)
```

```r
save(Deviance.sequenVaryLj, file = "toyExNNGPsequenVaryLjDeviance.RData")
spatpredVaryLj <- predictNewLocVaryLj(regVaryLj.simu.sequen, 9,
                                      distOrigNew, distNewNew,
                                      NewX = NULL, NewTrials = NULL, Verbose = TRUE,
                                      seed = 29)
save(spatpredVaryLj, file = "toyExNNGPspatpredVaryLj.RData")
tempppredVaryLj <- predictNewTime(regVaryLj.simu.sequen, (trainingT+1):Nu, seed = 29)
save(tempppredVaryLj, file = "toyExNNGPtempppredVaryLj.RData")
fittedClusGpMat <- matrix(0, m, 3)
clusVaryLj10 <- clusteringVaryLj(regVaryLj.simu.sequen, o = 1, nkeep = 10,
                                 nCent = numSpatOverallGroups)
fittedClusGpMat[,1] <- clusVaryLj10$cluster
clusVaryLj100 <- clusteringVaryLj(regVaryLj.simu.sequen, o = 1, nkeep = 100,
                                  nCent = numSpatOverallGroups)
fittedClusGpMat[,2] <- clusVaryLj100$cluster
clusVaryLj1000 <- clusteringVaryLj(regVaryLj.simu.sequen, o = 1, nkeep = 1000,
                                   nCent = numSpatOverallGroups)
fittedClusGpMat[,3] <- clusVaryLj1000$cluster
weightsNumIter <- c(10, 100, 1000)
colnames(fittedClusGpMat) <- paste(weightsNumIter, "iterWeights")
save(fittedClusGpMat, file = "NNGPsequenVaryLjtoyExfittedClusGpMat.RData")
library(spBFA) # to compare overall runtime, Diags, Deviances, and future time prediction results
reg.simu <- bfa_sp(Y ~ 0, data = dat, dist = Dtraining, time = Time,  K = K,
                   starting = NULL, hypers = NULL, tuning = NULL, mcmc = MCMC,
                   L = L,
                   family = "normal",
                   trials = NULL,
                   temporal.structure = "exponential",
                   spatial.structure = "continuous",
                   seed = 29,
                   gamma.shrinkage = TRUE,
                   include.space = TRUE,
                   clustering = TRUE)
save(reg.simu, file="toyExspBFAL10.RData")
Diags <- spBFA::diagnostics(reg.simu, diags = c("dic", "dinf", "waic"),
                            keepDeviance = TRUE)
save(Diags, file="toyExspBFAL10Diags.RData")
Deviance <- as.mcmc(Diags$deviance)
save(Deviance, file = "toyExspBFAL10Deviance.RData")
tempppredspBFAL10 <- predict(reg.simu, (trainingT+1):Nu, seed = 29) # from spBFA
save(tempppredspBFAL10, file = "toyExspBFAL10tempppred.RData")
reg.simu.LInf <- bfa_sp(Y ~ 0, data = dat, dist = Dtraining, time = Time,  K = K,
                   starting = NULL, hypers = NULL, tuning = NULL, mcmc = MCMC,
                   L = Inf,
                   family = "normal",
                   trials = NULL,
                   temporal.structure = "exponential",
                   spatial.structure = "continuous",
                   seed = 29,
                   gamma.shrinkage = TRUE,
                   include.space = TRUE,
                   clustering = TRUE)
save(reg.simu.LInf, file="toyExspBFALInf.RData")
```

```r
Diags.LInf <- spBFA::diagnostics(reg.simu.LInf, diags = c("dic", "dinf", "waic"),
                                 keepDeviance = TRUE)
save(Diags.LInf, file="toyExspBFALInfDiags.RData")
Deviance.LInf <- as.mcmc(Diags.LInf$deviance)
save(Deviance.LInf, file = "toyExspBFALInfDeviance.RData")
tempppredspBFALInf <- predict(reg.simu.LInf, (trainingT+1):Nu, seed = 29)
save(tempppredspBFALInf, file = "toyExspBFALInftemppred.RData")
```

Comparing spatempBFA and spBFA

Overall Model Fitting Time

```r
setwd(paste(projDirec, "simu/mainScalabilityVerificationSimu/toy example", sep = "/"))
setwd("spBFA")
load("toyExspBFAL10.RData"); reg.simu$runtime # 6.72 hours
```

```
## [1] "Model runtime: 6.72 hours"
```

```r
load("toyExspBFALInf.RData"); reg.simu.LInf$runtime # 8.21 hours
```

```
## [1] "Model runtime: 8.21 hours"
```

```r
load("toyExspBFAL10Diags.RData")
load("toyExspBFAL10Deviance.RData")
load("toyExspBFAL10temppred.RData")
load("toyExspBFALInfDiags.RData")
load("toyExspBFALInfDeviance.RData")
load("toyExspBFALInftemppred.RData")
spBFADiags <- Diags; spBFADeviance <- Deviance
spBFADiagsLInf <- Diags.LInf; spBFADevianceLInf <- Deviance.LInf
rm(Diags, Diags.LInf, Deviance, Deviance.LInf)
setwd("../fullGPfixedL")
load("toyExfullGPfixedL.RData"); regFixedL.simu$runtime # longer package version 1.71 hours
```

```
## [1] "Model runtime: 1.71 hours"
```

```r
load("toyExfullGPfixedLDiags.RData")
load("toyExfullGPfixedLDeviance.RData")
load("GibbsStepTimeFixedLfullGP.RData")
load("toyExNNGPtempppredFixedL.RData")
load("toyExNNGPspatpredFixedL.RData")
load("fullGPfixedLtoyExfittedClusGpMat.RData")
fittedClusGpMat.fullGPfixedL <- fittedClusGpMat
setwd("../NNGPblockFixedL")
load("toyExNNGPblockFixedL.RData"); regFixedL.simu.block$runtime # longer package version 1.7 hours
```

```
## [1] "Model runtime: 1.7 hours"
```

```r
load("toyExNNGPblockFixedLDiags.RData")
load("toyExNNGPblockFixedLDeviance.RData")
load("GibbsStepTimeFixedLblock.RData")
load("toyExNNGPtempppredFixedLblock.RData")
load("toyExNNGPspatpredFixedLblock.RData")
load("NNGPblockFixedLtoyExfittedClusGpMat.RData")
fittedClusGpMat.NNGPblockFixedL <- fittedClusGpMat
```

```r
setwd("../NNGPsequenFixedL")
load("toyExNNGPsequenFixedL.RData"); regFixedL.simu.sequen$runtime # longer package version 1.38 hours
```

```
## [1] "Model runtime: 1.38 hours"
```

```r
load("toyExNNGPsequenFixedLDiags.RData")
load("toyExNNGPsequenFixedLDeviance.RData")
load("GibbsStepTimeFixedLsequen.RData")
load("toyExNNGPtemppredFixedLsequen.RData")
load("toyExNNGPspatpredFixedLsequen.RData")
load("NNGPsequenFixedLtoyExfittedClusGpMat.RData")
fittedClusGpMat.NNGPsequenFixedL <- fittedClusGpMat
setwd("../NNGPsequenVaryLj")
load("toyExNNGPsequenVaryLj.RData"); regVaryLj.simu.sequen$runtime # longer package version 1.41 hours
```

```
## [1] "Model runtime: 1.41 hours"
```

```r
load("toyExNNGPsequenVaryLjDiags.RData")
load("toyExNNGPsequenVaryLjDeviance.RData")
load("GibbsStepTimeVaryLjsequen.RData")
load("toyExNNGPtemppredVaryLj.RData")
load("toyExNNGPspatpredVaryLj.RData")
load("NNGPsequenVaryLjtoyExfittedClusGpMat.RData")
fittedClusGpMat.NNGPsequenVaryLj <- fittedClusGpMat
rm(fittedClusGpMat)
rm(reg.simu, reg.simu.LInf)
rm(regFixedL.simu, regFixedL.simu.block, regFixedL.simu.sequen, regVaryLj.simu.sequen)
setwd("..")
```

As expected, our 4 models are a few times faster than the 2 `spBFA` models in terms of main model fitting.

```r
round(apply(GibbsStepTimeFixedLfullGP, 2, summary), digits = 3)
```

```
##              z      xi  theta delta  alpha kappa    rho    eta upsilon   psi
## Min.     32.00   3.000  3.000 0.000 27.000 2.000 15.000  3.000   0.000 0.000
## 1st Qu.  36.00   4.000  4.000 0.000 30.000 2.000 17.000  4.000   0.000 0.000
## Median   38.00   4.000  4.000 0.000 31.000 2.000 17.000  4.000   0.000 0.000
## Mean     44.45   4.686  4.318 0.002 34.403 2.188 19.702  4.449   0.001 0.002
## 3rd Qu.  50.00   5.000  5.000 0.000 36.000 2.000 22.000  5.000   0.000 0.000
## Max.    365.00 280.000 15.000 5.000 67.000 8.000 39.000 17.000   2.000 4.000
```

```r
round(apply(GibbsStepTimeFixedLblock, 2, summary), digits = 3)
```

```
##             z     xi  theta delta  alpha kappa    rho    eta upsilon psi
## Min.     44.0  3.000  3.000     0 24.000 1.000  6.000  3.000       0   0
## 1st Qu.  50.0  4.000  4.000     0 26.000 1.000  7.000  4.000       0   0
## Median   52.0  4.000  4.000     0 27.000 1.000  7.000  4.000       0   0
## Mean     57.8  4.352  4.199     0 28.965 1.006  7.503  4.424       0   0
## 3rd Qu.  55.0  4.000  4.000     0 28.000 1.000  7.000  5.000       0   0
## Max.    389.0 13.000 12.000     0 67.000 8.000 20.000 15.000       0   1
```

```r
round(apply(GibbsStepTimeFixedLsequen, 2, summary), digits = 3)
```

```
##               z     xi  theta delta  alpha kappa    rho    eta upsilon   psi
## Min.     39.000  3.000  3.000 0.000 10.000  1.00  6.000  3.000       0 0.000
## 1st Qu.  44.000  4.000  4.000 0.000 11.000  1.00  7.000  4.000       0 0.000
## Median   47.000  4.000  4.000 0.000 11.000  1.00  8.000  5.000       0 0.000
```

```
## Mean     55.377  4.781  4.466 0.001 13.206  1.17  8.602  4.786       0 0.002
## 3rd Qu.  69.000  6.000  5.000 0.000 16.000  1.00 10.000  5.000       0 0.000
## Max.    393.000 14.000 12.000 3.000 28.000  7.00 21.000 13.000       0 4.000
```

```
round(apply(GibbsStepTimeVaryLjSequen, 2, summary), digits = 3)
```

```
##         u     xi  theta delta   alpha kappa    rho    eta upsilon   psi
## Min.    0  1.000  3.000 0.000  49.000 1.000  6.000  3.000       0 0.000
## 1st Qu. 0  1.000  4.000 0.000  60.000 1.000  7.000  4.000       0 0.000
## Median  0  1.000  4.000 0.000  64.000 1.000  7.000  5.000       0 0.000
## Mean    0  1.686  4.234 0.001  73.114 1.018  8.175  4.767       0 0.002
## 3rd Qu. 0  2.000  5.000 0.000  85.000 1.000  9.000  5.000       0 0.000
## Max.    1 11.000 13.000 4.000 393.000 9.000 17.000 14.000       0 2.000
```

```
rm(GibbsStepTimeFixedLfullGP, GibbsStepTimeFixedLblock)
rm(GibbsStepTimeFixedLsequen, GibbsStepTimeVaryLjSequen)
```

The version of our `spatempBFA` package run for this toy example records Gibbs sampler time (in milliseconds) at each kept post-burn-in MCMC iteration for 10 key parameters $z_{jl}(\boldsymbol{s}_{i,o})$'s or $u_j(\boldsymbol{s}_{i,o})$'s, $\xi_j(\boldsymbol{s}_{i,o})$'s, θ_{jl_j}'s, $\delta_{1:k}$, ρ, κ, $\alpha_{jl_j}(\boldsymbol{s}_{i,o})$'s, ψ, Υ, and $\boldsymbol{\eta}_t$'s. Some summary statistics given above correspond well to their counterparts in Section 6.1's simulation experiments in terms of relative magnitudes.

- Compared to their `fullGPfixedL` counterparts, `NNGPblockFixedL`'s Gibbs sampler steps corresponding to ρ and κ are evidently accelerated by our spatial NNGP prior;
- The only Gibbs sampler step time that should clearly differ between `NNGPblockFixedL` and `NNGPsequenFixedL` is the step updating all $\alpha_{jl}(\boldsymbol{s}_{i,o})$'s, which result from whether we adopt our sequential updating method or not. Here, `NNGPsequenFixedL` is mostly more than two times faster than `NNGPblockFixedL` for the posterior sampling step corresponding to $\alpha_{jl}(\boldsymbol{s}_{i,o})$'s;
- Thanks to our slice sampling approach, `NNGPsequenVaryLj`'s Gibbs sampler steps for $u_j(\boldsymbol{s}_{i,o})$'s, $\xi_j(\boldsymbol{s}_{i,o})$'s, ρ, and κ are significantly faster than `NNGPsequenFixedL`'s Gibbs sampler steps for $z_{jl}(\boldsymbol{s}_{i,o})$'s, $\xi_j(\boldsymbol{s}_{i,o})$'s, ρ, and κ. It turns out that `NNGPsequenVaryLj`'s Gibbs sampler step for $\alpha_{jl_j}(\boldsymbol{s}_{i,o})$'s is slower than its `NNGPsequenFixedL` counterpart, indicating that inefficiencies caused by case discussion, calculating all required upper or lower bounds, and rejection sampling outweigh acceleration brought about by slice sampling's ensured non-increasing posterior samples for L_j's through the MCMC iterations;
- There aren't any significant differences between our 4 models regarding posterior sampling time for the 3 temporal parameters ψ, Υ, and $\boldsymbol{\eta}_t$'s.

Diagnostics and Posterior Deviances

```
print(Diags[setdiff(names(Diags), "deviance")])
```

```
## $dic
## $dic$dic
## [1] -17147.93
##
## $dic$pd
## [1] -162.3459
##
##
## $dinf
## $dinf$p
## [1] 888.1297
##
## $dinf$g
## [1] 174.849
##
```

```
## $dinf$dinf
## [1] 1062.979
##
##
## $meanIC
## $meanIC$postMSE
## [1] 0.06038643
##
## $meanIC$postVar
## [1] 0.05046192
##
##
## $waic
## $waic$waic
##           [,1]
## [1,] -6055.828
##
## $waic$p_waic
##          [,1]
## [1,] 5742.575
##
## $waic$lppd
## [1] 8770.489
##
## $waic$p_waic_1
## [1] 555.392
```

```r
print(Diags.block[setdiff(names(Diags.block), "deviance")])
```

```
## $dic
## $dic$dic
## [1] -17100.27
##
## $dic$pd
## [1] -115.5819
##
##
## $dinf
## $dinf$p
## [1] 888.151
##
## $dinf$g
## [1] 174.8288
##
## $dinf$dinf
## [1] 1062.98
##
##
## $meanIC
## $meanIC$postMSE
## [1] 0.06038649
##
## $meanIC$postVar
## [1] 0.05046312
##
```

```
##
## $waic
## $waic$waic
##           [,1]
## [1,] -6197.602
##
## $waic$p_waic
##          [,1]
## [1,] 5681.492
##
## $waic$lppd
## [1] 8780.293
##
## $waic$p_waic_1
## [1] 575.8981
```

```r
print(Diags.sequen[setdiff(names(Diags.sequen), "deviance")])
```

```
## $dic
## $dic$dic
## [1] -17106.77
##
## $dic$pd
## [1] -121.8139
##
##
## $dinf
## $dinf$p
## [1] 888.2227
##
## $dinf$g
## [1] 174.8333
##
## $dinf$dinf
## [1] 1063.056
##
##
## $meanIC
## $meanIC$postMSE
## [1] 0.06039082
##
## $meanIC$postVar
## [1] 0.0504672
##
##
## $waic
## $waic$waic
##           [,1]
## [1,] -6792.772
##
## $waic$p_waic
##        [,1]
## [1,] 5369.8
##
## $waic$lppd
```

```
## [1] 8766.186
##
## $waic$p_waic_1
## [1] 547.4167
```

```r
print(Diags.sequenVaryLj[setdiff(names(Diags.sequenVaryLj), "deviance")])
```

```
## $dic
## $dic$dic
## [1] -17108.78
##
## $dic$pd
## [1] -162.5282
##
##
## $dinf
## $dinf$p
## [1] 889.3638
##
## $dinf$g
## [1] 175.7445
##
## $dinf$dinf
## [1] 1065.108
##
##
## $meanIC
## $meanIC$postMSE
## [1] 0.06050741
##
## $meanIC$postVar
## [1] 0.05053203
##
##
## $waic
## $waic$waic
##           [,1]
## [1,] -6256.451
##
## $waic$p_waic
##          [,1]
## [1,] 5586.045
##
## $waic$lppd
## [1] 8714.271
##
## $waic$p_waic_1
## [1] 482.2889
```

```r
print(spBFADiags[setdiff(names(spBFADiags), "deviance")])
```

```
## $dic
## $dic$dic
## [1] -17233.1
##
```

```
## $dic$pd
## [1] -247.847
##
##
## $dinf
## $dinf$p
## [1] 888.2564
##
## $dinf$g
## [1] 174.5164
##
## $dinf$dinf
## [1] 1062.773
##
##
## $waic
## $waic$waic
## [1] -5963.637
##
## $waic$p_waic
## [1] 5759.678
##
## $waic$lppd
## [1] 8741.497
##
## $waic$p_waic_1
## [1] 497.7424
```

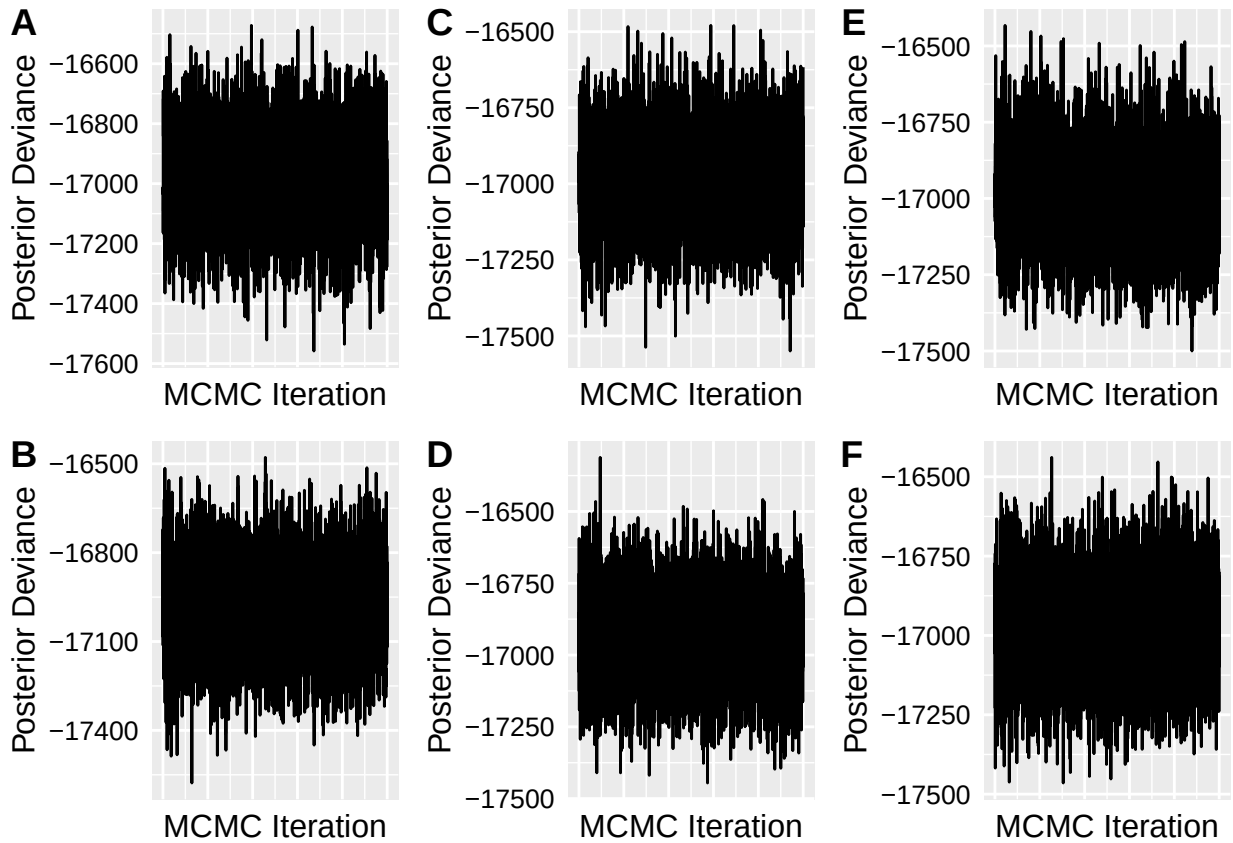
```r
print(spBFADiagsLInf[setdiff(names(spBFADiagsLInf), "deviance")])
```

```
## $dic
## $dic$dic
## [1] -17094.82
##
## $dic$pd
## [1] -122.3814
##
##
## $dinf
## $dinf$p
## [1] 889.1091
##
## $dinf$g
## [1] 174.3595
##
## $dinf$dinf
## [1] 1063.469
##
##
## $waic
## $waic$waic
## [1] -6057.219
##
## $waic$p_waic
## [1] 5695.098
```

```
##
## $waic$lppd
## [1] 8723.707
##
## $waic$p_waic_1
## [1] 474.9774
```

```r
rm(Diags, Diags.block, Diags.sequen, Diags.sequenVaryLj)
rm(spBFADiags, spBFADiagsLInf)
NKeep <- 5000
postDeviancesDF <- data.frame(MCMCiter = 1:NKeep, fullGPfixedL = as.vector(Deviance),
                              NNGPblockFixedL = as.vector(Deviance.block),
                              NNGPsequenFixedL = as.vector(Deviance.sequen),
                              NNGPsequenVaryLj = as.vector(Deviance.sequenVaryLj),
                              spBFAL10 = as.vector(spBFADeviance),
                              spBFALInf = as.vector(spBFADevianceLInf))
rm(Deviance, Deviance.block, Deviance.sequen, Deviance.sequenVaryLj)
rm(spBFADeviance, spBFADevianceLInf)
fullGPfixedLpostDeviances <- ggplot(postDeviancesDF) +
  geom_line(aes(x = MCMCiter, y = fullGPfixedL)) +
  labs(x = "MCMC Iteration", y = "Posterior Deviance") +
  theme(axis.title.x = element_text(size = 12),
        axis.title.y = element_text(size = 12),
        axis.text.x = element_blank(),
        axis.text.y = element_text(size = 10, color = "black"),
        axis.ticks.x = element_blank(), axis.ticks.y = element_blank())
NNGPblockFixedLpostDeviances <- ggplot(postDeviancesDF) +
  geom_line(aes(x = MCMCiter, y = NNGPblockFixedL)) +
  labs(x = "MCMC Iteration", y = "Posterior Deviance") +
  theme(axis.title.x = element_text(size = 12),
        axis.title.y = element_text(size = 12),
        axis.text.x = element_blank(),
        axis.text.y = element_text(size = 10, color = "black"),
        axis.ticks.x = element_blank(), axis.ticks.y = element_blank())
NNGPsequenFixedLpostDeviances <- ggplot(postDeviancesDF) +
  geom_line(aes(x = MCMCiter, y = NNGPsequenFixedL)) +
  labs(x = "MCMC Iteration", y = "Posterior Deviance") +
  theme(axis.title.x = element_text(size = 12),
        axis.title.y = element_text(size = 12),
        axis.text.x = element_blank(),
        axis.text.y = element_text(size = 10, color = "black"),
        axis.ticks.x = element_blank(), axis.ticks.y = element_blank())
NNGPsequenVaryLjpostDeviances <- ggplot(postDeviancesDF) +
  geom_line(aes(x = MCMCiter, y = NNGPsequenVaryLj)) +
  labs(x = "MCMC Iteration", y = "Posterior Deviance") +
  theme(axis.title.x = element_text(size = 12),
        axis.title.y = element_text(size = 12),
        axis.text.x = element_blank(),
        axis.text.y = element_text(size = 10, color = "black"),
        axis.ticks.x = element_blank(), axis.ticks.y = element_blank())
spBFAL10Deviances <- ggplot(postDeviancesDF) +
  geom_line(aes(x = MCMCiter, y = spBFAL10)) +
  labs(x = "MCMC Iteration", y = "Posterior Deviance") +
  theme(axis.title.x = element_text(size = 12),
```

```
        axis.title.y = element_text(size = 12),
        axis.text.x = element_blank(),
        axis.text.y = element_text(size = 10, color = "black"),
        axis.ticks.x = element_blank(), axis.ticks.y = element_blank())
spBFALInfDeviances <- ggplot(postDeviancesDF) +
  geom_line(aes(x = MCMCiter, y = spBFALInf)) +
  labs(x = "MCMC Iteration", y = "Posterior Deviance") +
  theme(axis.title.x = element_text(size = 12),
        axis.title.y = element_text(size = 12),
        axis.text.x = element_blank(),
        axis.text.y = element_text(size = 10, color = "black"),
        axis.ticks.x = element_blank(), axis.ticks.y = element_blank())
ggarrange(fullGPfixedLpostDeviances, NNGPsequenFixedLpostDeviances, spBFAL10Deviances,
          NNGPblockFixedLpostDeviances, NNGPsequenVaryLjpostDeviances, spBFALInfDeviances,
          labels = c("A", "C", "E", "B", "D", "F"), align = "v",
          ncol = 3, nrow = 2)
```




```
#ggsave("toyExPostDeviancesSmall.png", width = 24, height = 16, units = "cm")
rm(fullGPfixedLpostDeviances, NNGPblockFixedLpostDeviances)
rm(NNGPsequenFixedLpostDeviances, NNGPsequenVaryLjpostDeviances)
rm(postDeviancesDF, spBFAL10Deviances, spBFALInfDeviances)
```

All 6 models' diagnostics and converged posterior deviances are comparable.

Predictions for all 352 Training Locations at the 10 Future Time Points

```
summary(YtestingTemp)
```

```
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
## -9.70426 -2.25231  0.16317 -0.08109  2.94192  7.71861
```

```
summary(abs(YtestingTemp))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.01934 0.64301 2.79260 3.25421 4.54703 9.70426
```

```
YtestingTempDF <- data.frame(YtestingTemp = YtestingTemp)
ggplot(YtestingTempDF) + geom_histogram(aes(x = YtestingTemp), bins = 50) +
  labs(y = "Count", x = "Actual Future Time y") +
  theme(axis.ticks.x = element_blank(), axis.ticks.y = element_blank(),
        plot.title = element_text(hjust = 0.5)) +
  ggtitle("Actual Distribution of y at All 352 Training Locations and Time 51:60")
```

```
tempPredMetricMat <- matrix(0, 3, 6,
                            dimnames = list(c("postMeanMSE", "postMSE", "postVar"),
                                            c("fullGPfixedL", "NNGPblockFixedL",
                                              "NNGPsequenFixedL",
                                              "NNGPsequenVaryLj",
                                              "spBFAL10", "spBFALInf")))
# testingT = 10
ytempPredList = tempppredFixedL$Y
ytempPred <- t(matrix(unlist(ytempPredList), ncol = testingT * m * O))
```

```r
# (testingT x m x O) x NKeep
ytempPredMean <- apply(ytempPred, 1, mean)
# of length N = testingT x O x m (rowMeans) note that O = 1 here
tempPredMetricMat[1, 1] <- mean((ytempPredMean - ytempPred)^2)
diffMat <- sweep(ytempPred, 1, YtestingTemp, "-")
tempPredMetricMat[2, 1] = mean(rowMeans(diffMat^2))
tempPredMetricMat[3, 1] = mean(apply(ytempPred, 1, var))
ytempPredList = tempppredFixedLblock$Y
ytempPred <- t(matrix(unlist(ytempPredList), ncol = testingT * m * O))
# (testingT x m x O) x NKeep
ytempPredMean <- apply(ytempPred, 1, mean)
# of length N = testingT x O x m (rowMeans) note that O = 1 here
tempPredMetricMat[1, 2] <- mean((ytempPredMean - ytempPred)^2)
diffMat <- sweep(ytempPred, 1, YtestingTemp, "-")
tempPredMetricMat[2, 2] = mean(rowMeans(diffMat^2))
tempPredMetricMat[3, 2] = mean(apply(ytempPred, 1, var))
ytempPredList = tempppredFixedLsequen$Y
ytempPred <- t(matrix(unlist(ytempPredList), ncol = testingT * m * O))
# (testingT x m x O) x NKeep
ytempPredMean <- apply(ytempPred, 1, mean)
# of length N = testingT x O x m (rowMeans) note that O = 1 here
tempPredMetricMat[1, 3] <- mean((ytempPredMean - ytempPred)^2)
diffMat <- sweep(ytempPred, 1, YtestingTemp, "-")
tempPredMetricMat[2, 3] = mean(rowMeans(diffMat^2))
tempPredMetricMat[3, 3] = mean(apply(ytempPred, 1, var))
ytempPredList = tempppredVaryLj$Y
ytempPred <- t(matrix(unlist(ytempPredList), ncol = testingT * m * O))
# (testingT x m x O) x NKeep
ytempPredMean <- apply(ytempPred, 1, mean)
# of length N = testingT x O x m (rowMeans) note that O = 1 here
tempPredMetricMat[1, 4] <- mean((ytempPredMean - ytempPred)^2)
diffMat <- sweep(ytempPred, 1, YtestingTemp, "-")
tempPredMetricMat[2, 4] = mean(rowMeans(diffMat^2))
tempPredMetricMat[3, 4] = mean(apply(ytempPred, 1, var))
ytempPredList = tempppredspBFAL10$Y
ytempPred <- t(matrix(unlist(ytempPredList), ncol = testingT * m * O))
# (testingT x m x O) x NKeep
ytempPredMean <- apply(ytempPred, 1, mean)
# of length N = testingT x O x m (rowMeans) note that O = 1 here
tempPredMetricMat[1, 5] <- mean((ytempPredMean - ytempPred)^2)
diffMat <- sweep(ytempPred, 1, YtestingTemp, "-")
tempPredMetricMat[2, 5] = mean(rowMeans(diffMat^2))
tempPredMetricMat[3, 5] = mean(apply(ytempPred, 1, var))
ytempPredList = tempppredspBFALInf$Y
ytempPred <- t(matrix(unlist(ytempPredList), ncol = testingT * m * O)) # (testingT x m x O) x NKeep
ytempPredMean <- apply(ytempPred, 1, mean)
# of length N = testingT x O x m (rowMeans) note that O = 1 here
tempPredMetricMat[1, 6] <- mean((ytempPredMean - ytempPred)^2)
diffMat <- sweep(ytempPred, 1, YtestingTemp, "-")
tempPredMetricMat[2, 6] = mean(rowMeans(diffMat^2))
tempPredMetricMat[3, 6] = mean(apply(ytempPred, 1, var))
rm(tempppredFixedL, tempppredFixedLblock, tempppredFixedLsequen, tempppredVaryLj)
tempPredMetricMat
```

```
##             fullGPfixedL NNGPblockFixedL NNGPsequenFixedL NNGPsequenVaryLj
## postMeanMSE     12.97098        17.03785         10.13314         88.29886
## postMSE         31.02957        35.05762         28.20416        106.10105
## postVar         12.97357        17.04126         10.13516         88.31652
##             spBFAL10 spBFALInf
## postMeanMSE 10.92390  11.56887
## postMSE     29.16116  29.90661
## postVar     10.92608  11.57118
```

```
rm(tempppredspBFAL10, tempppredspBFALInf)
```

Temporal prediction results from our models `NNGPsequenFixedL` and `fullGPfixedL` are comparable to their `spBFA` model counterparts.

Spatial Prediction and Clustering by `spatempBFA`

Predictions for all 50 Training Time Points at the 9 Testing Locations

Spatial Prediction Metrics

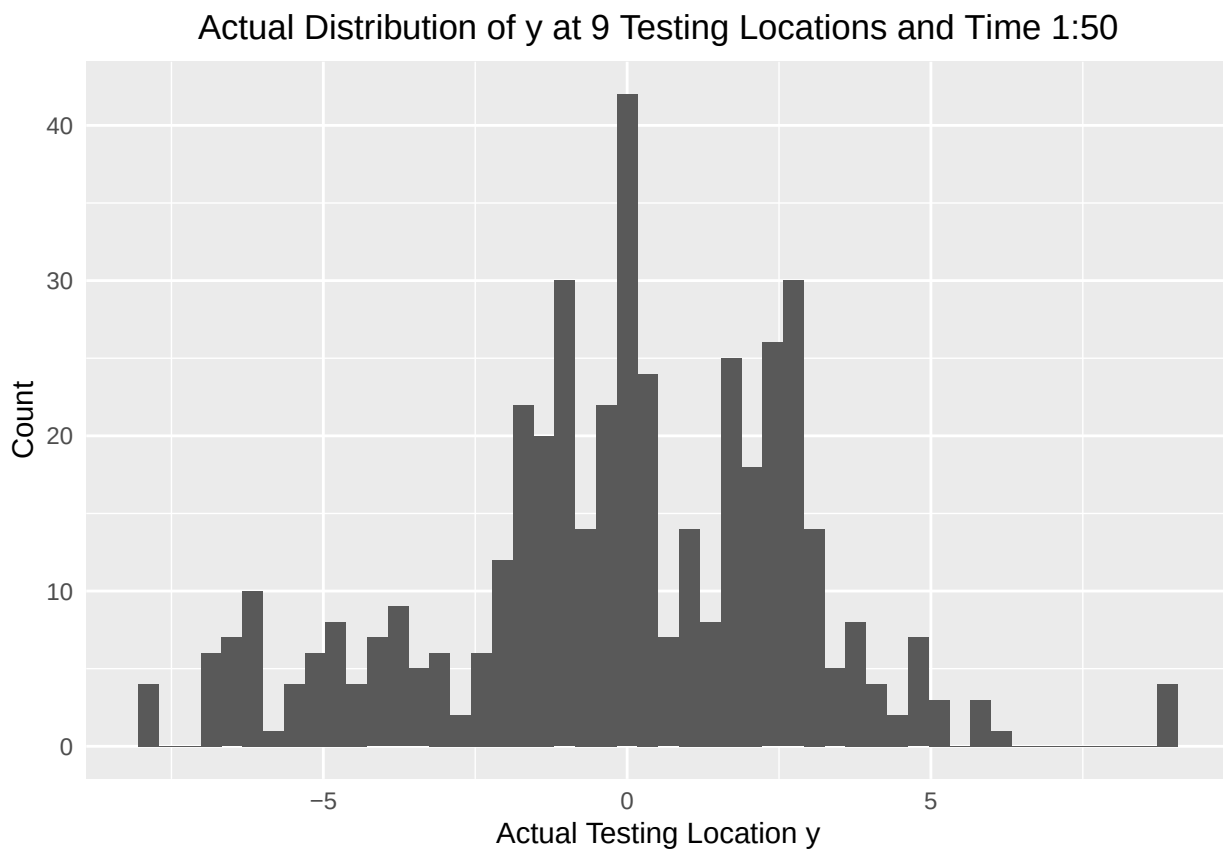
```
summary(YtestingSpat)
```

```
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
## -7.97873 -1.58654 -0.01463 -0.14828  2.18089  8.79778
```

```
summary(abs(YtestingSpat))
```

```
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
## 0.001286 0.921419 1.858349 2.308221 3.078693 8.797785
```

```
YtestingSpatDF <- data.frame(YtestingSpat = YtestingSpat)
ggplot(YtestingSpatDF) + geom_histogram(aes(x = YtestingSpat), bins = 50) +
  labs(y = "Count", x = "Actual Testing Location y") +
  theme(axis.ticks.x = element_blank(), axis.ticks.y = element_blank(),
        plot.title = element_text(hjust = 0.5)) +
  ggtitle("Actual Distribution of y at 9 Testing Locations and Time 1:50")
```




```
spatPredMetricMat <- matrix(0, 3, 4,
                            dimnames = list(c("postMeanMSE", "postMSE", "postVar"),
                                            c("fullGPfixedL", "NNGPblockFixedL",
                                              "NNGPsequenFixedL", "NNGPsequenVaryLj")))
testingLocNum <- M - m
ylocPredList = spatpredFixedL$Y
ylocPred <- t(matrix(unlist(ylocPredList), ncol = testingLocNum * trainingT * O))
#(testingLocNum x trainingT x O) x NKeep
ylocPredMean <- apply(ylocPred, 1, mean)
# of length N = testingLocNum x O x trainingT (rowMeans) note that O = 1 here
spatPredMetricMat[1, 1] <- mean((ylocPredMean - YtestingSpat)^2)
diffMat <- sweep(ylocPred, 1, YtestingSpat, "-")
spatPredMetricMat[2, 1] = mean(rowMeans(diffMat^2))
spatPredMetricMat[3, 1] = mean(apply(ylocPred, 1, var))
ylocPredList = spatpredFixedLblock$Y
ylocPred <- t(matrix(unlist(ylocPredList), ncol = testingLocNum * trainingT * O))
#(testingLocNum x trainingT x O) x NKeep
ylocPredMean <- apply(ylocPred, 1, mean)
# of length N = testingLocNum x O x trainingT (rowMeans) note that O = 1 here
spatPredMetricMat[1, 2] <- mean((ylocPredMean - YtestingSpat)^2)
diffMat <- sweep(ylocPred, 1, YtestingSpat, "-")
spatPredMetricMat[2, 2] = mean(rowMeans(diffMat^2))
spatPredMetricMat[3, 2] = mean(apply(ylocPred, 1, var))
ylocPredList = spatpredFixedLsequen$Y
ylocPred <- t(matrix(unlist(ylocPredList), ncol = testingLocNum * trainingT * O))
#(testingLocNum x trainingT x O) x NKeep
```

```r
ylocPredMean <- apply(ylocPred, 1, mean)
# of length N = testingLocNum x O x trainingT (rowMeans) note that O = 1 here
spatPredMetricMat[1, 3] <- mean((ylocPredMean - YtestingSpat)^2)
diffMat <- sweep(ylocPred, 1, YtestingSpat, "-")
spatPredMetricMat[2, 3] = mean(rowMeans(diffMat^2))
spatPredMetricMat[3, 3] = mean(apply(ylocPred, 1, var))
ylocPredList = spatpredVaryLj$Y
ylocPred <- t(matrix(unlist(ylocPredList), ncol = testingLocNum * trainingT * O))
#(testingLocNum x trainingT x O) x NKeep
ylocPredMean <- apply(ylocPred, 1, mean)
# of length N = testingLocNum x O x trainingT (rowMeans) note that O = 1 here
spatPredMetricMat[1, 4] <- mean((ylocPredMean - YtestingSpat)^2)
diffMat <- sweep(ylocPred, 1, YtestingSpat, "-")
spatPredMetricMat[2, 4] = mean(rowMeans(diffMat^2))
spatPredMetricMat[3, 4] = mean(apply(ylocPred, 1, var))
spatPredMetricMat
```

```
##             fullGPfixedL NNGPblockFixedL NNGPsequenFixedL NNGPsequenVaryLj
## postMeanMSE     1.692678        1.629900         1.763877         2.723069
## postMSE        11.621003       11.506880        11.799492        13.235839
## postVar         9.930311        9.878956        10.037622        10.514873
```

All spatial prediction metrics are appropriately small in light of the magnitudes of the actual $y_t(\boldsymbol{s}_i)$'s at the 9 testing locations and time $t = 1, 2, \ldots, 50$.

Spatial Prediction Time Breakdown for Our 4 Models

```r
spatPredKrigTime <- matrix(0, 2, 4,
                           dimnames = list(c("alpha", "weightsXiLambda"),
                                           c("fullGPfixedL", "NNGPblockFixedL",
                                             "NNGPsequenFixedL", "NNGPsequenVaryLj")))
spatPredKrigTime[1, 1] = spatpredFixedL$alphaKrigTime
spatPredKrigTime[2, 1] = spatpredFixedL$weightsXiLambdaKrigTime
spatPredKrigTime[1, 2] = spatpredFixedLblock$alphaKrigTime
spatPredKrigTime[2, 2] = spatpredFixedLblock$weightsXiLambdaKrigTime
spatPredKrigTime[1, 3] = spatpredFixedLsequen$alphaKrigTime
spatPredKrigTime[2, 3] = spatpredFixedLsequen$weightsXiLambdaKrigTime
spatPredKrigTime[1, 4] = spatpredVaryLj$alphaKrigTime
spatPredKrigTime[2, 4] = spatpredVaryLj$weightsXiLambdaKrigTime
spatPredKrigTime
```

```
##                 fullGPfixedL NNGPblockFixedL NNGPsequenFixedL NNGPsequenVaryLj
## alpha                  95940           74773            65935            30307
## weightsXiLambda          645            1179             1007              224
```

The results correspond well to what we have derived in Appendix H.

- `alphaKrigTime` corresponding to `NNGPblockFixedL` and `NNGPsequenFixedL` should be close and markedly smaller than that of `fullGPfixedL`. `NNGPsequenVaryLj`'s `alphaKrigTime` should be even smaller;
- `weightsXiLambdaKrigTime` corresponding to `NNGPsequenVaryLj` should be smaller than their counterparts from the other 3 models.

It is worth noting that the breakdown time (in milliseconds) above can be quite different with the same set-up. We present below results from some other spatial prediction instances from the same model fitting objects we have obtained.

```r
setwd("fullGPfixedL")
load("toyExfullGPfixedL.RData")
spatpredFixedL <- predictNewLocFixedL(regFixedL.simu, 9, distOrigNew, distNewNew,
                                      NewX = NULL, NewTrials = NULL, Verbose = TRUE,
                                      seed = 29)
save(spatpredFixedL, file = "s29toyExNNGPspatpredFixedL.RData")
spatpredFixedL <- predictNewLocFixedL(regFixedL.simu, 9, distOrigNew, distNewNew,
                                      NewX = NULL, NewTrials = NULL, Verbose = TRUE,
                                      seed = 27)
save(spatpredFixedL, file = "s27toyExNNGPspatpredFixedL.RData")
spatpredFixedL <- predictNewLocFixedL(regFixedL.simu, 9, distOrigNew, distNewNew,
                                      NewX = NULL, NewTrials = NULL, Verbose = TRUE,
                                      seed = 31)
save(spatpredFixedL, file = "s31toyExNNGPspatpredFixedL.RData")
spatpredFixedL <- predictNewLocFixedL(regFixedL.simu, 9, distOrigNew, distNewNew,
                                      NewX = NULL, NewTrials = NULL, Verbose = TRUE,
                                      seed = 19)
save(spatpredFixedL, file = "s19toyExNNGPspatpredFixedL.RData")
setwd("../NNGPblockFixedL")
load("toyExNNGPblockFixedL.RData")
spatpredFixedLblock <- predictNewLocFixedL(regFixedL.simu.block, 9,
                                           distOrigNew, distNewNew,
                                           NewX = NULL, NewTrials = NULL, Verbose = TRUE,
                                           seed = 29)
save(spatpredFixedLblock, file = "s29toyExNNGPspatpredFixedLblock.RData")
spatpredFixedLblock <- predictNewLocFixedL(regFixedL.simu.block, 9,
                                           distOrigNew, distNewNew,
                                           NewX = NULL, NewTrials = NULL, Verbose = TRUE,
                                           seed = 27)
save(spatpredFixedLblock, file = "s27toyExNNGPspatpredFixedLblock.RData")
spatpredFixedLblock <- predictNewLocFixedL(regFixedL.simu.block, 9,
                                           distOrigNew, distNewNew,
                                           NewX = NULL, NewTrials = NULL, Verbose = TRUE,
                                           seed = 31)
save(spatpredFixedLblock, file = "s31toyExNNGPspatpredFixedLblock.RData")
spatpredFixedLblock <- predictNewLocFixedL(regFixedL.simu.block, 9,
                                           distOrigNew, distNewNew,
                                           NewX = NULL, NewTrials = NULL, Verbose = TRUE,
                                           seed = 19)
save(spatpredFixedLblock, file = "s19toyExNNGPspatpredFixedLblock.RData")
setwd("../NNGPsequenFixedL")
load("toyExNNGPsequenFixedL.RData")
spatpredFixedLsequen <- predictNewLocFixedL(regFixedL.simu.sequen, 9,
                                            distOrigNew, distNewNew,
                                            NewX = NULL, NewTrials = NULL, Verbose = TRUE,
                                            seed = 29)
save(spatpredFixedLsequen, file = "s29toyExNNGPspatpredFixedLsequen.RData")
spatpredFixedLsequen <- predictNewLocFixedL(regFixedL.simu.sequen, 9,
                                            distOrigNew, distNewNew,
                                            NewX = NULL, NewTrials = NULL, Verbose = TRUE,
                                            seed = 27)
save(spatpredFixedLsequen, file = "s27toyExNNGPspatpredFixedLsequen.RData")
spatpredFixedLsequen <- predictNewLocFixedL(regFixedL.simu.sequen, 9,
```

```r
                                          distOrigNew, distNewNew,
                                          NewX = NULL, NewTrials = NULL, Verbose = TRUE,
                                          seed = 31)
save(spatpredFixedLsequen, file = "s31toyExNNGPspatpredFixedLsequen.RData")
spatpredFixedLsequen <- predictNewLocFixedL(regFixedL.simu.sequen, 9,
                                          distOrigNew, distNewNew,
                                          NewX = NULL, NewTrials = NULL, Verbose = TRUE,
                                          seed = 19)
save(spatpredFixedLsequen, file = "s19toyExNNGPspatpredFixedLsequen.RData")
setwd("../NNGPsequenVaryLj")
load("toyExNNGPsequenVaryLj.RData")
spatpredVaryLj <- predictNewLocVaryLj(regVaryLj.simu.sequen, 9, distOrigNew, distNewNew,
                                      NewX = NULL, NewTrials = NULL, Verbose = TRUE,
                                      seed = 29)
save(spatpredVaryLj, file = "s29toyExNNGPspatpredVaryLj.RData")
spatpredVaryLj <- predictNewLocVaryLj(regVaryLj.simu.sequen, 9, distOrigNew, distNewNew,
                                      NewX = NULL, NewTrials = NULL, Verbose = TRUE,
                                      seed = 27)
save(spatpredVaryLj, file = "s27toyExNNGPspatpredVaryLj.RData")
spatpredVaryLj <- predictNewLocVaryLj(regVaryLj.simu.sequen, 9, distOrigNew, distNewNew,
                                      NewX = NULL, NewTrials = NULL, Verbose = TRUE,
                                      seed = 31)
save(spatpredVaryLj, file = "s31toyExNNGPspatpredVaryLj.RData")
spatpredVaryLj <- predictNewLocVaryLj(regVaryLj.simu.sequen, 9, distOrigNew, distNewNew,
                                      NewX = NULL, NewTrials = NULL, Verbose = TRUE,
                                      seed = 19)
save(spatpredVaryLj, file = "s19toyExNNGPspatpredVaryLj.RData")
```

```r
setwd("fullGPfixedL")
load("s29toyExNNGPspatpredFixedL.RData")
setwd("../NNGPblockFixedL")
load("s29toyExNNGPspatpredFixedLblock.RData")
setwd("../NNGPsequenFixedL")
load("s29toyExNNGPspatpredFixedLsequen.RData")
setwd("../NNGPsequenVaryLj")
load("s29toyExNNGPspatpredVaryLj.RData")
setwd("..")
spatPredKrigTime <- matrix(0, 2, 4,
                           dimnames = list(c("alpha", "weightsXiLambda"),
                                           c("fullGPfixedL", "NNGPblockFixedL",
                                             "NNGPsequenFixedL", "NNGPsequenVaryLj")))
spatPredKrigTime[1, 1] = spatpredFixedL$alphaKrigTime
spatPredKrigTime[2, 1] = spatpredFixedL$weightsXiLambdaKrigTime
spatPredKrigTime[1, 2] = spatpredFixedLblock$alphaKrigTime
spatPredKrigTime[2, 2] = spatpredFixedLblock$weightsXiLambdaKrigTime
spatPredKrigTime[1, 3] = spatpredFixedLsequen$alphaKrigTime
spatPredKrigTime[2, 3] = spatpredFixedLsequen$weightsXiLambdaKrigTime
spatPredKrigTime[1, 4] = spatpredVaryLj$alphaKrigTime
spatPredKrigTime[2, 4] = spatpredVaryLj$weightsXiLambdaKrigTime
spatPredKrigTime
```

```
##                 fullGPfixedL NNGPblockFixedL NNGPsequenFixedL NNGPsequenVaryLj
## alpha                  95400           23610            28375            25023
## weightsXiLambda          379             419              426              261
```

```r
spatPredMetricMat <- matrix(0, 3, 4,
                            dimnames = list(c("postMeanMSE", "postMSE", "postVar"),
                                            c("fullGPfixedL", "NNGPblockFixedL",
                                              "NNGPsequenFixedL", "NNGPsequenVaryLj")))
testingLocNum <- M - m
ylocPredList = spatpredFixedL$Y
ylocPred <- t(matrix(unlist(ylocPredList), ncol = testingLocNum * trainingT * O))
#(testingLocNum x trainingT x O) x NKeep
ylocPredMean <- apply(ylocPred, 1, mean)
# of length N = testingLocNum x O x trainingT (rowMeans) note that O = 1 here
spatPredMetricMat[1, 1] <- mean((ylocPredMean - YtestingSpat)^2)
diffMat <- sweep(ylocPred, 1, YtestingSpat, "-")
spatPredMetricMat[2, 1] = mean(rowMeans(diffMat^2))
spatPredMetricMat[3, 1] = mean(apply(ylocPred, 1, var))
ylocPredList = spatpredFixedLblock$Y
ylocPred <- t(matrix(unlist(ylocPredList), ncol = testingLocNum * trainingT * O))
#(testingLocNum x trainingT x O) x NKeep
ylocPredMean <- apply(ylocPred, 1, mean)
# of length N = testingLocNum x O x trainingT (rowMeans) note that O = 1 here
spatPredMetricMat[1, 2] <- mean((ylocPredMean - YtestingSpat)^2)
diffMat <- sweep(ylocPred, 1, YtestingSpat, "-")
spatPredMetricMat[2, 2] = mean(rowMeans(diffMat^2))
spatPredMetricMat[3, 2] = mean(apply(ylocPred, 1, var))
ylocPredList = spatpredFixedLsequen$Y
ylocPred <- t(matrix(unlist(ylocPredList), ncol = testingLocNum * trainingT * O))
#(testingLocNum x trainingT x O) x NKeep
ylocPredMean <- apply(ylocPred, 1, mean)
# of length N = testingLocNum x O x trainingT (rowMeans) note that O = 1 here
spatPredMetricMat[1, 3] <- mean((ylocPredMean - YtestingSpat)^2)
diffMat <- sweep(ylocPred, 1, YtestingSpat, "-")
spatPredMetricMat[2, 3] = mean(rowMeans(diffMat^2))
spatPredMetricMat[3, 3] = mean(apply(ylocPred, 1, var))
ylocPredList = spatpredVaryLj$Y
ylocPred <- t(matrix(unlist(ylocPredList), ncol = testingLocNum * trainingT * O))
#(testingLocNum x trainingT x O) x NKeep
ylocPredMean <- apply(ylocPred, 1, mean)
# of length N = testingLocNum x O x trainingT (rowMeans) note that O = 1 here
spatPredMetricMat[1, 4] <- mean((ylocPredMean - YtestingSpat)^2)
diffMat <- sweep(ylocPred, 1, YtestingSpat, "-")
spatPredMetricMat[2, 4] = mean(rowMeans(diffMat^2))
spatPredMetricMat[3, 4] = mean(apply(ylocPred, 1, var))
spatPredMetricMat
```

```
##             fullGPfixedL NNGPblockFixedL NNGPsequenFixedL NNGPsequenVaryLj
## postMeanMSE     1.692678        1.629900         1.763877         2.723069
## postMSE        11.621003       11.506880        11.799492        13.235839
## postVar         9.930311        9.878956        10.037622        10.514873
```

```r
setwd("fullGPfixedL")
load("s27toyExNNGPspatpredFixedL.RData")
setwd("../NNGPblockFixedL")
load("s27toyExNNGPspatpredFixedLblock.RData")
setwd("../NNGPsequenFixedL")
load("s27toyExNNGPspatpredFixedLsequen.RData")
```

```r
setwd("../NNGPsequenVaryLj")
load("s27toyExNNGPspatpredVaryLj.RData")
setwd("..")
spatPredKrigTime <- matrix(0, 2, 4,
                           dimnames = list(c("alpha", "weightsXiLambda"),
                                           c("fullGPfixedL", "NNGPblockFixedL",
                                             "NNGPsequenFixedL", "NNGPsequenVaryLj")))
spatPredKrigTime[1, 1] = spatpredFixedL$alphaKrigTime
spatPredKrigTime[2, 1] = spatpredFixedL$weightsXiLambdaKrigTime
spatPredKrigTime[1, 2] = spatpredFixedLblock$alphaKrigTime
spatPredKrigTime[2, 2] = spatpredFixedLblock$weightsXiLambdaKrigTime
spatPredKrigTime[1, 3] = spatpredFixedLsequen$alphaKrigTime
spatPredKrigTime[2, 3] = spatpredFixedLsequen$weightsXiLambdaKrigTime
spatPredKrigTime[1, 4] = spatpredVaryLj$alphaKrigTime
spatPredKrigTime[2, 4] = spatpredVaryLj$weightsXiLambdaKrigTime
spatPredKrigTime
```

```
##                 fullGPfixedL NNGPblockFixedL NNGPsequenFixedL NNGPsequenVaryLj
## alpha                  89444           23571            28068            24830
## weightsXiLambda          370             399              380              234
```

```r
spatPredMetricMat <- matrix(0, 3, 4,
                           dimnames = list(c("postMeanMSE", "postMSE", "postVar"),
                                           c("fullGPfixedL", "NNGPblockFixedL",
                                             "NNGPsequenFixedL", "NNGPsequenVaryLj")))
testingLocNum <- M - m
ylocPredList = spatpredFixedL$Y
ylocPred <- t(matrix(unlist(ylocPredList), ncol = testingLocNum * trainingT * O))
#(testingLocNum x trainingT x O) x NKeep
ylocPredMean <- apply(ylocPred, 1, mean)
# of length N = testingLocNum x O x trainingT (rowMeans) note that O = 1 here
spatPredMetricMat[1, 1] <- mean((ylocPredMean - YtestingSpat)^2)
diffMat <- sweep(ylocPred, 1, YtestingSpat, "-")
spatPredMetricMat[2, 1] = mean(rowMeans(diffMat^2))
spatPredMetricMat[3, 1] = mean(apply(ylocPred, 1, var))
ylocPredList = spatpredFixedLblock$Y
ylocPred <- t(matrix(unlist(ylocPredList), ncol = testingLocNum * trainingT * O))
#(testingLocNum x trainingT x O) x NKeep
ylocPredMean <- apply(ylocPred, 1, mean)
# of length N = testingLocNum x O x trainingT (rowMeans) note that O = 1 here
spatPredMetricMat[1, 2] <- mean((ylocPredMean - YtestingSpat)^2)
diffMat <- sweep(ylocPred, 1, YtestingSpat, "-")
spatPredMetricMat[2, 2] = mean(rowMeans(diffMat^2))
spatPredMetricMat[3, 2] = mean(apply(ylocPred, 1, var))
ylocPredList = spatpredFixedLsequen$Y
ylocPred <- t(matrix(unlist(ylocPredList), ncol = testingLocNum * trainingT * O))
#(testingLocNum x trainingT x O) x NKeep
ylocPredMean <- apply(ylocPred, 1, mean)
# of length N = testingLocNum x O x trainingT (rowMeans) note that O = 1 here
spatPredMetricMat[1, 3] <- mean((ylocPredMean - YtestingSpat)^2)
diffMat <- sweep(ylocPred, 1, YtestingSpat, "-")
spatPredMetricMat[2, 3] = mean(rowMeans(diffMat^2))
spatPredMetricMat[3, 3] = mean(apply(ylocPred, 1, var))
ylocPredList = spatpredVaryLj$Y
```

```r
ylocPred <- t(matrix(unlist(ylocPredList), ncol = testingLocNum * trainingT * O))
#(testingLocNum x trainingT x O) x NKeep
ylocPredMean <- apply(ylocPred, 1, mean)
# of length N = testingLocNum x O x trainingT (rowMeans) note that O = 1 here
spatPredMetricMat[1, 4] <- mean((ylocPredMean - YtestingSpat)^2)
diffMat <- sweep(ylocPred, 1, YtestingSpat, "-")
spatPredMetricMat[2, 4] = mean(rowMeans(diffMat^2))
spatPredMetricMat[3, 4] = mean(apply(ylocPred, 1, var))
spatPredMetricMat
```

```
##             fullGPfixedL NNGPblockFixedL NNGPsequenFixedL NNGPsequenVaryLj
## postMeanMSE     1.714656        1.636480         1.794968         2.827745
## postMSE        10.064480        9.973663        10.280365        11.848705
## postVar         8.351494        8.338850         8.487095         9.022764
```

```r
setwd("fullGPfixedL")
load("s31toyExNNGPspatpredFixedL.RData")
setwd("../NNGPblockFixedL")
load("s31toyExNNGPspatpredFixedLblock.RData")
setwd("../NNGPsequenFixedL")
load("s31toyExNNGPspatpredFixedLsequen.RData")
setwd("../NNGPsequenVaryLj")
load("s31toyExNNGPspatpredVaryLj.RData")
setwd("..")
spatPredKrigTime <- matrix(0, 2, 4,
                           dimnames = list(c("alpha", "weightsXiLambda"),
                                           c("fullGPfixedL", "NNGPblockFixedL",
                                             "NNGPsequenFixedL", "NNGPsequenVaryLj")))
spatPredKrigTime[1, 1] = spatpredFixedL$alphaKrigTime
spatPredKrigTime[2, 1] = spatpredFixedL$weightsXiLambdaKrigTime
spatPredKrigTime[1, 2] = spatpredFixedLblock$alphaKrigTime
spatPredKrigTime[2, 2] = spatpredFixedLblock$weightsXiLambdaKrigTime
spatPredKrigTime[1, 3] = spatpredFixedLsequen$alphaKrigTime
spatPredKrigTime[2, 3] = spatpredFixedLsequen$weightsXiLambdaKrigTime
spatPredKrigTime[1, 4] = spatpredVaryLj$alphaKrigTime
spatPredKrigTime[2, 4] = spatpredVaryLj$weightsXiLambdaKrigTime
spatPredKrigTime
```

```
##                 fullGPfixedL NNGPblockFixedL NNGPsequenFixedL NNGPsequenVaryLj
## alpha                  88988           23804            28267            24964
## weightsXiLambda          406             396              392              230
```

```r
spatPredMetricMat <- matrix(0, 3, 4,
                            dimnames = list(c("postMeanMSE", "postMSE", "postVar"),
                                            c("fullGPfixedL", "NNGPblockFixedL",
                                              "NNGPsequenFixedL", "NNGPsequenVaryLj")))
testingLocNum <- M - m
ylocPredList = spatpredFixedL$Y
ylocPred <- t(matrix(unlist(ylocPredList), ncol = testingLocNum * trainingT * O))
#(testingLocNum x trainingT x O) x NKeep
ylocPredMean <- apply(ylocPred, 1, mean)
# of length N = testingLocNum x O x trainingT (rowMeans) note that O = 1 here
spatPredMetricMat[1, 1] <- mean((ylocPredMean - YtestingSpat)^2)
diffMat <- sweep(ylocPred, 1, YtestingSpat, "-")
spatPredMetricMat[2, 1] = mean(rowMeans(diffMat^2))
```

```r
spatPredMetricMat[3, 1] = mean(apply(ylocPred, 1, var))
ylocPredList = spatpredFixedLblock$Y
ylocPred <- t(matrix(unlist(ylocPredList), ncol = testingLocNum * trainingT * O))
#(testingLocNum x trainingT x O) x NKeep
ylocPredMean <- apply(ylocPred, 1, mean)
# of length N = testingLocNum x O x trainingT (rowMeans) note that O = 1 here
spatPredMetricMat[1, 2] <- mean((ylocPredMean - YtestingSpat)^2)
diffMat <- sweep(ylocPred, 1, YtestingSpat, "-")
spatPredMetricMat[2, 2] = mean(rowMeans(diffMat^2))
spatPredMetricMat[3, 2] = mean(apply(ylocPred, 1, var))
ylocPredList = spatpredFixedLsequen$Y
ylocPred <- t(matrix(unlist(ylocPredList), ncol = testingLocNum * trainingT * O))
#(testingLocNum x trainingT x O) x NKeep
ylocPredMean <- apply(ylocPred, 1, mean)
# of length N = testingLocNum x O x trainingT (rowMeans) note that O = 1 here
spatPredMetricMat[1, 3] <- mean((ylocPredMean - YtestingSpat)^2)
diffMat <- sweep(ylocPred, 1, YtestingSpat, "-")
spatPredMetricMat[2, 3] = mean(rowMeans(diffMat^2))
spatPredMetricMat[3, 3] = mean(apply(ylocPred, 1, var))
ylocPredList = spatpredVaryLj$Y
ylocPred <- t(matrix(unlist(ylocPredList), ncol = testingLocNum * trainingT * O))
#(testingLocNum x trainingT x O) x NKeep
ylocPredMean <- apply(ylocPred, 1, mean)
# of length N = testingLocNum x O x trainingT (rowMeans) note that O = 1 here
spatPredMetricMat[1, 4] <- mean((ylocPredMean - YtestingSpat)^2)
diffMat <- sweep(ylocPred, 1, YtestingSpat, "-")
spatPredMetricMat[2, 4] = mean(rowMeans(diffMat^2))
spatPredMetricMat[3, 4] = mean(apply(ylocPred, 1, var))
spatPredMetricMat
```

```
##             fullGPfixedL NNGPblockFixedL NNGPsequenFixedL NNGPsequenVaryLj
## postMeanMSE     1.633710        1.645057         1.823608         2.825989
## postMSE         9.850945        9.854366        10.180693        15.395982
## postVar         8.218879        8.210951         8.358756        12.572508
```

```r
setwd("fullGPfixedL")
load("s19toyExNNGPspatpredFixedL.RData")
setwd("../NNGPblockFixedL")
load("s19toyExNNGPspatpredFixedLblock.RData")
setwd("../NNGPsequenFixedL")
load("s19toyExNNGPspatpredFixedLsequen.RData")
setwd("../NNGPsequenVaryLj")
load("s19toyExNNGPspatpredVaryLj.RData")
setwd("..")
spatPredKrigTime <- matrix(0, 2, 4,
                           dimnames = list(c("alpha", "weightsXiLambda"),
                                           c("fullGPfixedL", "NNGPblockFixedL",
                                             "NNGPsequenFixedL", "NNGPsequenVaryLj")))
spatPredKrigTime[1, 1] = spatpredFixedL$alphaKrigTime
spatPredKrigTime[2, 1] = spatpredFixedL$weightsXiLambdaKrigTime
spatPredKrigTime[1, 2] = spatpredFixedLblock$alphaKrigTime
spatPredKrigTime[2, 2] = spatpredFixedLblock$weightsXiLambdaKrigTime
spatPredKrigTime[1, 3] = spatpredFixedLsequen$alphaKrigTime
spatPredKrigTime[2, 3] = spatpredFixedLsequen$weightsXiLambdaKrigTime
```

```r
spatPredKrigTime[1, 4] = spatpredVaryLj$alphaKrigTime
spatPredKrigTime[2, 4] = spatpredVaryLj$weightsXiLambdaKrigTime
spatPredKrigTime
```

```
##                 fullGPfixedL NNGPblockFixedL NNGPsequenFixedL NNGPsequenVaryLj
## alpha                  93321           24214            28429            25642
## weightsXiLambda          370             401              389              234
```

```r
spatPredMetricMat <- matrix(0, 3, 4,
                            dimnames = list(c("postMeanMSE", "postMSE", "postVar"),
                                            c("fullGPfixedL", "NNGPblockFixedL",
                                              "NNGPsequenFixedL", "NNGPsequenVaryLj")))
testingLocNum <- M - m
ylocPredList = spatpredFixedL$Y
ylocPred <- t(matrix(unlist(ylocPredList), ncol = testingLocNum * trainingT * O))
#(testingLocNum x trainingT x O) x NKeep
ylocPredMean <- apply(ylocPred, 1, mean)
# of length N = testingLocNum x O x trainingT (rowMeans) note that O = 1 here
spatPredMetricMat[1, 1] <- mean((ylocPredMean - YtestingSpat)^2)
diffMat <- sweep(ylocPred, 1, YtestingSpat, "-")
spatPredMetricMat[2, 1] = mean(rowMeans(diffMat^2))
spatPredMetricMat[3, 1] = mean(apply(ylocPred, 1, var))
ylocPredList = spatpredFixedLblock$Y
ylocPred <- t(matrix(unlist(ylocPredList), ncol = testingLocNum * trainingT * O))
#(testingLocNum x trainingT x O) x NKeep
ylocPredMean <- apply(ylocPred, 1, mean)
# of length N = testingLocNum x O x trainingT (rowMeans) note that O = 1 here
spatPredMetricMat[1, 2] <- mean((ylocPredMean - YtestingSpat)^2)
diffMat <- sweep(ylocPred, 1, YtestingSpat, "-")
spatPredMetricMat[2, 2] = mean(rowMeans(diffMat^2))
spatPredMetricMat[3, 2] = mean(apply(ylocPred, 1, var))
ylocPredList = spatpredFixedLsequen$Y
ylocPred <- t(matrix(unlist(ylocPredList), ncol = testingLocNum * trainingT * O))
#(testingLocNum x trainingT x O) x NKeep
ylocPredMean <- apply(ylocPred, 1, mean)
# of length N = testingLocNum x O x trainingT (rowMeans) note that O = 1 here
spatPredMetricMat[1, 3] <- mean((ylocPredMean - YtestingSpat)^2)
diffMat <- sweep(ylocPred, 1, YtestingSpat, "-")
spatPredMetricMat[2, 3] = mean(rowMeans(diffMat^2))
spatPredMetricMat[3, 3] = mean(apply(ylocPred, 1, var))
ylocPredList = spatpredVaryLj$Y
ylocPred <- t(matrix(unlist(ylocPredList), ncol = testingLocNum * trainingT * O))
#(testingLocNum x trainingT x O) x NKeep
ylocPredMean <- apply(ylocPred, 1, mean)
# of length N = testingLocNum x O x trainingT (rowMeans) note that O = 1 here
spatPredMetricMat[1, 4] <- mean((ylocPredMean - YtestingSpat)^2)
diffMat <- sweep(ylocPred, 1, YtestingSpat, "-")
spatPredMetricMat[2, 4] = mean(rowMeans(diffMat^2))
spatPredMetricMat[3, 4] = mean(apply(ylocPred, 1, var))
spatPredMetricMat
```

```
##             fullGPfixedL NNGPblockFixedL NNGPsequenFixedL NNGPsequenVaryLj
## postMeanMSE     1.828309        1.738902          1.86795         2.786246
## postMSE       130.331967      130.168763        130.43213        14.121343
```

```
## postVar       128.529364      128.455552        128.58990        11.337364
```

Spatial Clustering of Temporal Trends for all 352 Training Spatial Locations

```
calcRandIndex <- function(predictedCluster, actualGroup, numObs){
  denominator <- numObs * (numObs-1) / 2
  numerator <- denominator
  for (i in 1:(numObs-1)){
    for (j in (i + 1):numObs){
      if ( (predictedCluster[i]==predictedCluster[j]) + (actualGroup[i]==actualGroup[j]) == 1)
        numerator = numerator - 1
    }
  }
  randIndex <- numerator / denominator
  return(randIndex)
}
calcAccuRatio <- function(predictedCluster, actualGroup, numObs){
  actualGroupsPoss1 = factor(actualGroup, labels=c(1,2))
  actualGroupsPoss2 = factor(actualGroup, labels=c(2,1))
  accuRatio <- max(sum(predictedCluster==actualGroupsPoss1),
                   sum(predictedCluster==actualGroupsPoss2))/numObs
  return(accuRatio)
}
fittedClusGpMat <- cbind(fittedClusGpMat.fullGPfixedL, fittedClusGpMat.NNGPblockFixedL,
                         fittedClusGpMat.NNGPsequenFixedL, fittedClusGpMat.NNGPsequenVaryLj)
modelVec <- c("fullGPfixedL", "NNGPblockFixedL", "NNGPsequenFixedL", "NNGPsequenVaryLj")
clusIterWeightsVec <- colnames(fittedClusGpMat.fullGPfixedL)
# paste(c(10, 100, 1000), "iterWeights", sep = " ")
accuRatioMat <- matrix(apply(fittedClusGpMat, 2, calcAccuRatio,
                             actualGroup = spatGroupOverallTraining,
                             numObs = m), 3, 4,
                       dimnames = list(clusIterWeightsVec, modelVec))
accuRatioMat
```

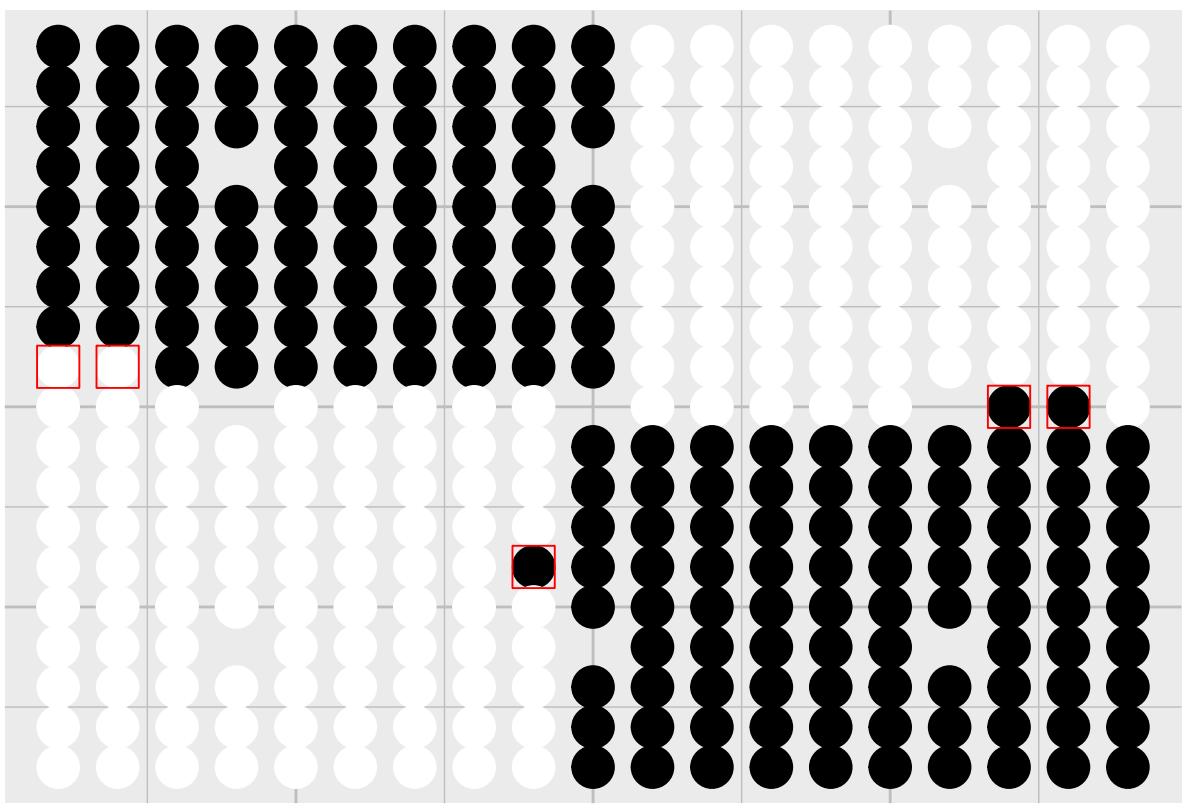
```
##                  fullGPfixedL NNGPblockFixedL NNGPsequenFixedL NNGPsequenVaryLj
## 10 iterWeights              1               1                1        0.9857955
## 100 iterWeights             1               1                1        0.9886364
## 1000 iterWeights            1               1                1        0.9886364
```

```
randIndMat <- matrix(apply(fittedClusGpMat, 2, calcRandIndex,
                           actualGroup = spatGroupOverallTraining,
                           numObs = m), 3, 4,
                     dimnames = list(clusIterWeightsVec, modelVec))
randIndMat
```

```
##                  fullGPfixedL NNGPblockFixedL NNGPsequenFixedL NNGPsequenVaryLj
## 10 iterWeights              1               1                1        0.9719147
## 100 iterWeights             1               1                1        0.9774670
## 1000 iterWeights            1               1                1        0.9774670
```

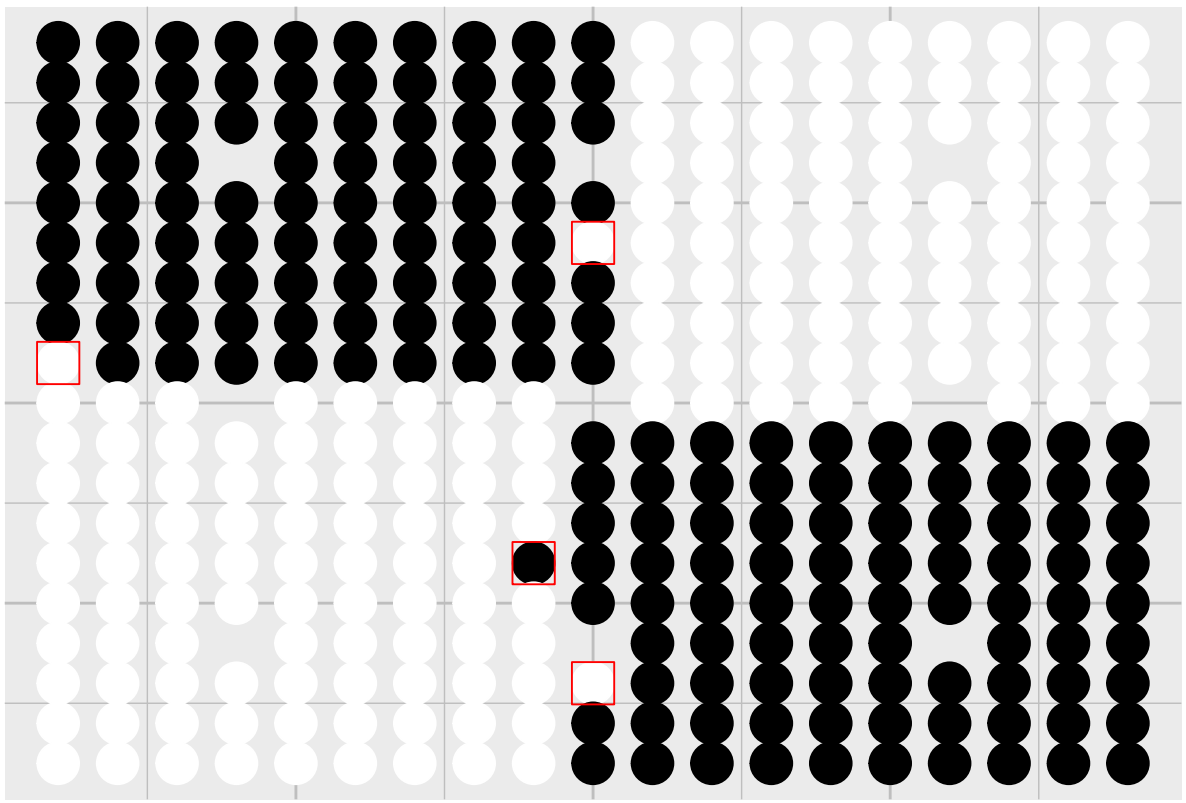
The above results suggest that our 3 models `fullGPfixedL`, `NNGPblockFixedL`, and `NNGPsequenFixedL` all lead to completely correct spatial clustering outcomes. `NNGPsequenVaryLj` k-means clustering on three posterior weights matrices formulated from 10, 100, and 1000 selected equally dispersed kept post-burn-in MCMC iterations' parameter estimates (Section 5.2) also leads to highly accurate spatial clustering results (with only 5, 4, and 4 out of 352 testing locations clustered wrongly), as depicted below.

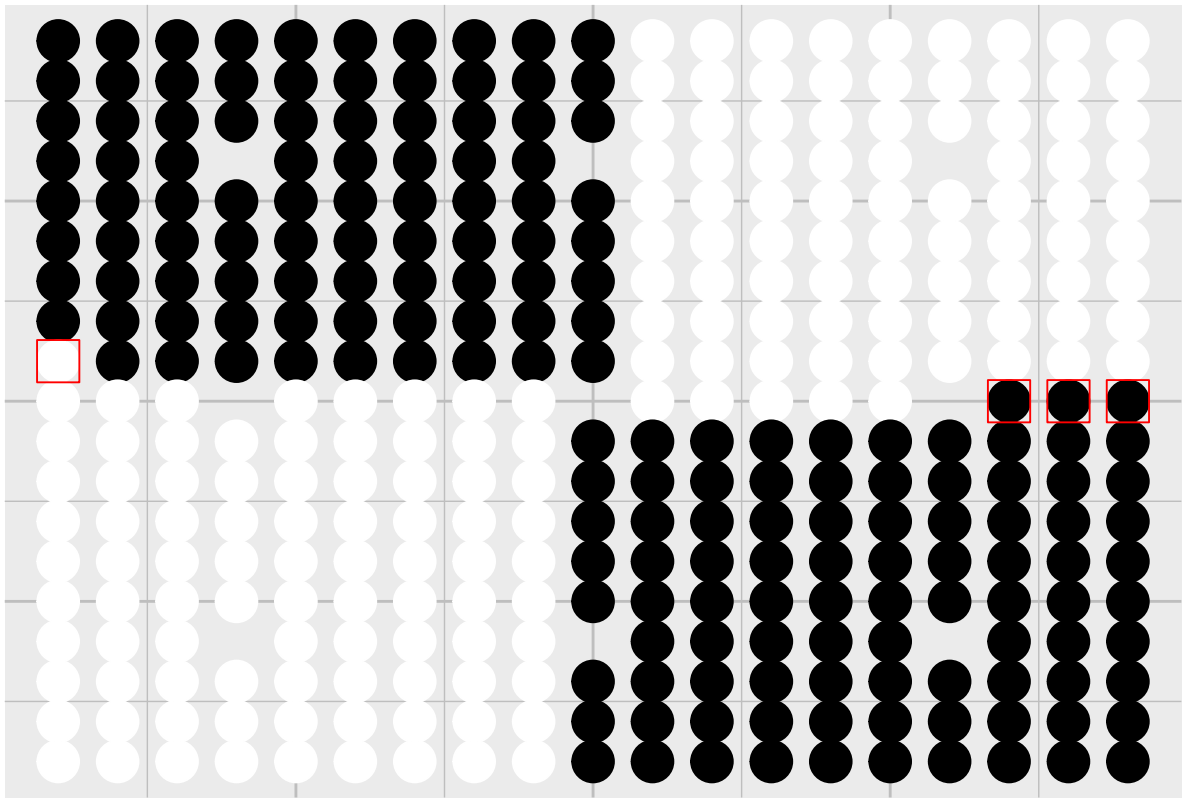
```r
xcoordTraining <- xcoord[-whichTesting]; ycoordTraining = ycoord[-whichTesting]
fittedNNGPsequenVaryLjspatGpDF <- data.frame(x = xcoordTraining, y = ycoordTraining,
                                             nkeep10 =
                                               as.factor(fittedClusGpMat.NNGPsequenVaryLj[,1]),
                                             nkeep100 =
                                               as.factor(fittedClusGpMat.NNGPsequenVaryLj[,2]),
                                             nkeep1000 =
                                               as.factor(fittedClusGpMat.NNGPsequenVaryLj[,3]))
nkeep10NNGPsequenVaryLjBW <- ggplot(fittedNNGPsequenVaryLjspatGpDF) +
  geom_point(aes(x = x, y = y, col = nkeep10), size = 7, show.legend = FALSE) +
  scale_color_manual(values = c("white", "black")) + labs(x = "", y = "") +
  theme(axis.text.x = element_blank(), axis.text.y = element_blank(),
        axis.ticks.x = element_blank(), axis.ticks.y = element_blank(),
        panel.grid.major = element_line(color="grey"),
        panel.grid.minor = element_line(color="grey"))
nkeep10agreements <- spatGroupOverallTraining + fittedClusGpMat.NNGPsequenVaryLj[,1] == 3
# 5 locations clustered wrongly (0.9857955*352 = 347)
disaDFnkeep10 <- data.frame(x = xcoordTraining[!nkeep10agreements],
                            y = ycoordTraining[!nkeep10agreements])
nkeep10NNGPsequenVaryLj <- nkeep10NNGPsequenVaryLjBW +
  geom_point(data = disaDFnkeep10, aes(x = x, y = y),
             col = "red", fill = NA, shape = 0, size = 7.1)
nkeep10NNGPsequenVaryLj
```



```r
nkeep100NNGPsequenVaryLjBW <- ggplot(fittedNNGPsequenVaryLjspatGpDF) +
  geom_point(aes(x = x, y = y, col = nkeep100), size = 7, show.legend = FALSE) +
```

```
  scale_color_manual(values = c("black", "white")) + labs(x = "", y = "") +
  theme(axis.text.x = element_blank(), axis.text.y = element_blank(),
        axis.ticks.x = element_blank(), axis.ticks.y = element_blank(),
        panel.grid.major = element_line(color="grey"),
        panel.grid.minor = element_line(color="grey"))
nkeep100agreements <- spatGroupOverallTraining == fittedClusGpMat.NNGPsequenVaryLj[,2]
# 4 locations clustered wrongly (0.9886364*352 = 348)
disaDFnkeep100 <- data.frame(x = xcoordTraining[!nkeep100agreements],
                             y = ycoordTraining[!nkeep100agreements])
nkeep100NNGPsequenVaryLj <- nkeep100NNGPsequenVaryLjBW +
  geom_point(data = disaDFnkeep100, aes(x = x, y = y),
             col = "red", fill = NA, shape = 0, size = 7.1)
nkeep100NNGPsequenVaryLj
```



```
nkeep1000NNGPsequenVaryLjBW <- ggplot(fittedNNGPsequenVaryLjspatGpDF) +
  geom_point(aes(x = x, y = y, col = nkeep1000), size = 7, show.legend = FALSE) +
  scale_color_manual(values = c("white", "black")) + labs(x = "", y = "") +
  theme(axis.text.x = element_blank(), axis.text.y = element_blank(),
        axis.ticks.x = element_blank(), axis.ticks.y = element_blank(),
        panel.grid.major = element_line(color="grey"),
        panel.grid.minor = element_line(color="grey"))
nkeep1000agreements <- spatGroupOverallTraining + fittedClusGpMat.NNGPsequenVaryLj[,3] == 3
# 4 locations clustered wrongly (0.9886364*352 = 348)
disaDFnkeep1000 <- data.frame(x = xcoordTraining[!nkeep1000agreements],
                              y = ycoordTraining[!nkeep1000agreements])
nkeep1000NNGPsequenVaryLj <- nkeep1000NNGPsequenVaryLjBW +
```

```
  geom_point(data = disaDFnkeep1000, aes(x = x, y = y),
             col = "red", fill = NA, shape = 0, size = 7.1)
nkeep1000NNGPsequenVaryLj
```